

UNICA
UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAIfer Lab
Joint lab on Safety and Security of AI

Decision trees

Giorgio Fumera, Ambra Demontis, Maura Pintor
Pattern Recognition and Applications Lab
Department of Electrical and Electronic Engineering
University of Cagliari (Italy)

Artificial Intelligence, academic year 2025/2026

Decision Trees

Machine learning

Decision Trees

Given a classification problem with d attributes $x_1, \ldots, x_d$ with discrete and finite domains $X_1, \ldots, X_d$ (e.g., Boolean attributes), a Decision Tree (DT) is a tree graph that represents a mutually exclusive set of classification rules of the form:

IF *condition* THEN *class*

where

- *condition* is a Boolean expression (*test*) consisting of the conjunction (AND) of one or more conditions of the form $x_i = v_i$, where $i \in \{1, \ldots, d\}$ and $v_i \in X_i$
- *class* is the class label predicted by the DT for any instance which fulfills *condition*

A problem that can be addressed with decision trees

Consider an e-mail spam filtering problem with:

- class labels $Y = \{\text{Spam}, \text{Legitimate}\}$
- $d = 5$ Boolean attributes, denoting the occurrence of the following terms in an input e-mail: `cheap, buy, bargain, university, conference`.

Example:

Subject: **Cheap** flights this summer!
Hello, **buy** your trip to Spain now: a real **bargain** price, only this week.

A problem that can be addressed with decision trees

Example:

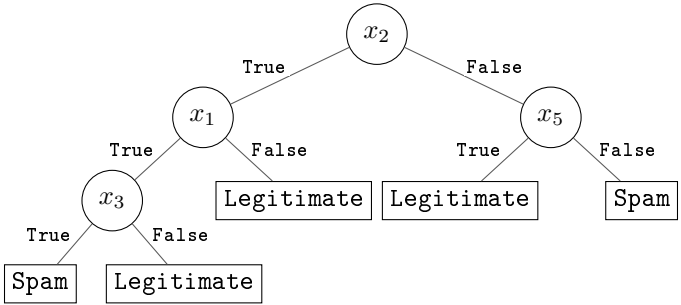
Subject: **Cheap** flights this summer!
Hello, **buy** your trip to Spain now: a real **bargain** price, only this week.

Attribute	Term	Example
x_1	cheap	True
x_2	buy	True
x_3	bargain	True
x_4	university	False
x_5	conference	False

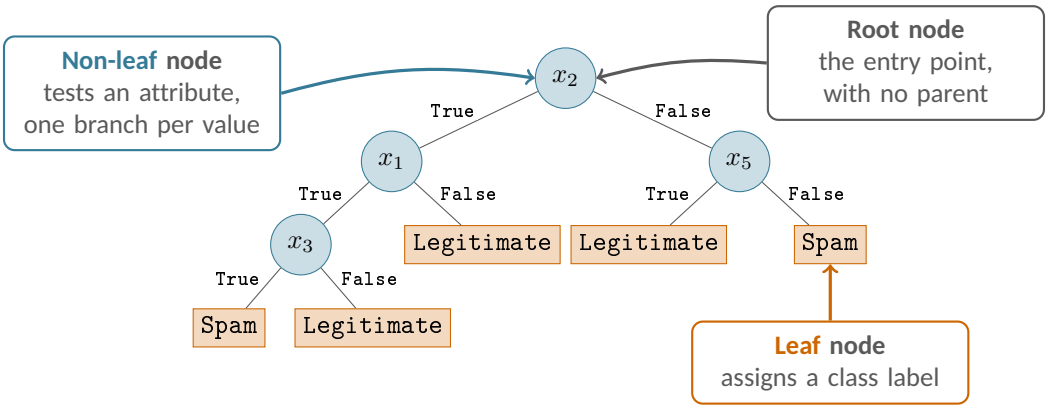
$$\mathbf{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_5 \end{pmatrix} = \begin{pmatrix} \text{True} \\ \text{True} \\ \text{True} \\ \text{False} \\ \text{False} \end{pmatrix}$$

Decision Trees: an example

The following is a possible DT (not necessarily an accurate one):

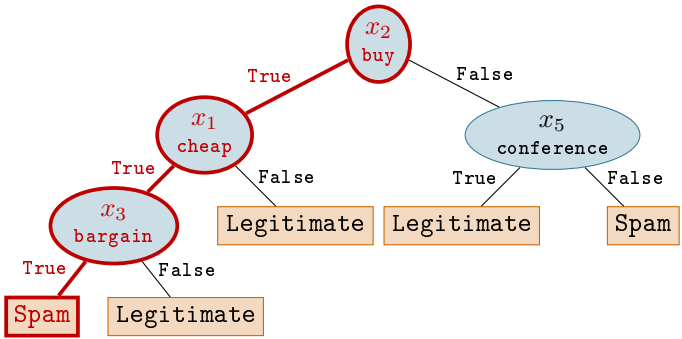


Decision Trees: leaf and non-leaf nodes



Decision Trees: an example (e-mail classification)

We can then find the decision based on the evaluation of the single nodes.

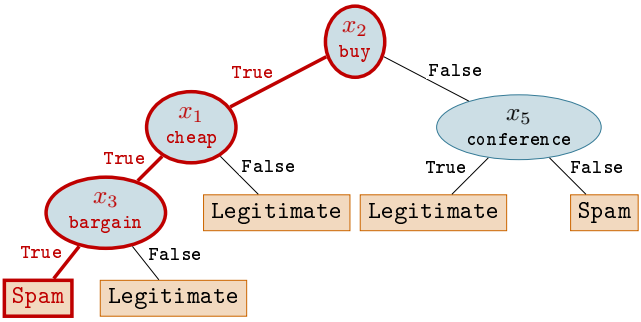


Decision Trees

Formally:

- every non-leaf node tests one attribute x_i, with one **edge** for each value in its domain X_i
- every leaf node has one class label (different leaves can share it)
- every **path** from the **root** to a leaf is one classification rule
- an attribute appears at most once in a **path** (but may appear multiple times in the same DT)

Decision Trees: an example of path



The classification rule represented by the leftmost path can be written as:

$$\text{IF } x_2 = \texttt{True} \text{ AND } x_1 = \texttt{True} \text{ AND } x_3 = \texttt{True} \text{ THEN } \texttt{spam}$$

(read as: if the terms `buy`, `cheap` and `bargain` appear in an e-mail, then it is `spam`)

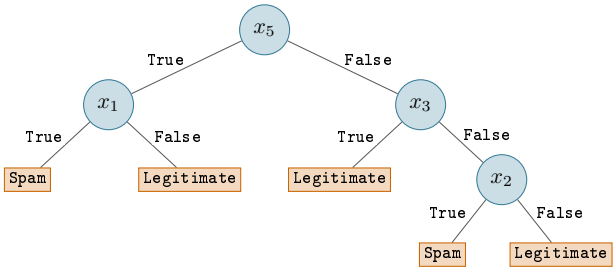
Decision Trees: exercise

Consider the following e-mail and the DT on the right. Which class does the DT assign to the e-mail?

Side question: do you think the decision is correct?

> Subject: **Free** seminar on Friday
>
> Hi, please **click** the link below to register. The **meeting** room is still to be confirmed.

Attribute	Term
x_1	free
x_2	winner
x_3	meeting
x_4	deadline
x_5	click



Decision Trees

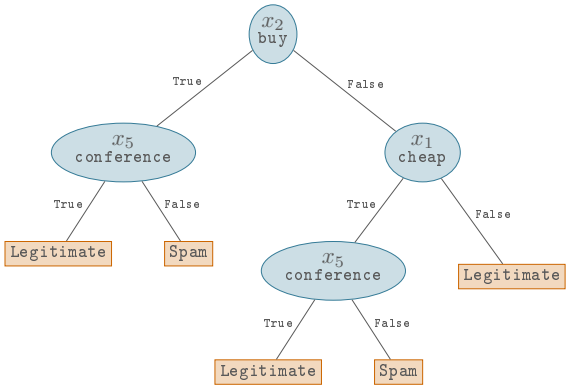
From the above properties, the classification rules of a DT are **complete** and **mutually exclusive**, i.e., exactly one of them applies to any given feature vector.

Some attributes may also not appear in a DT, i.e., the corresponding attributes are not taken into account for any decision rule.

Every attribute may appear in different paths, i.e., in different classification rules (e.g., the root node appears in all rules).

Decision Trees: another an example

An example of DT with an attribute (x_5) that appears in two different paths.



Note also that x_3 and x_4 do not appear in the above DT.

Toward a learning algorithm for Decision Trees

The hypothesis space (classifier model) H is the set of all possible, distinct DTs that can be constructed using the class labels Y and the attribute space X.

The **goal** of the learning algorithm is to select the *simplest*[1] DT $h \in H$, *consistent*[2] with a training set T.

To rigorously define the above goal, we need to measure the "complexity" of a DT.

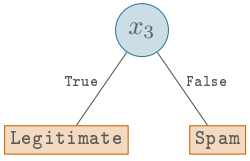
[1] Inductive inference: a logical process where you move from specific observations to broad, general conclusions.

[2] A DT is *consistent* with the training set T if it classifies correctly all the examples in T.

Toward a learning algorithm for Decision Trees

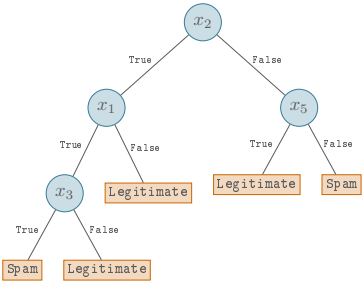
A possible measure of the complexity of a DT is the number of its non-leaf nodes.

Complexity: 1



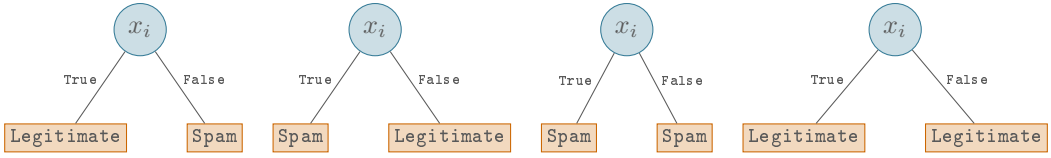
One-level DTs are called **decision stumps**

Complexity: 4



Toward a learning algorithm for Decision Trees

Let's see how to find the smallest DT consistent with T in the previous example (remember that d is the number of attributes).
First build all DTs of size 1, then all DTs of size 2, and so on, until a consistent DT is found.



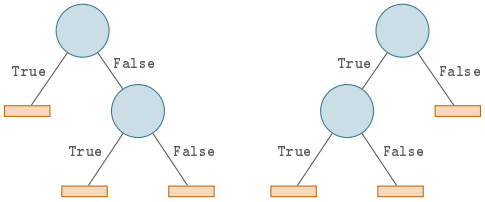
This leads to the following number of possible decision stumps[3]:

$$4 \times d \qquad \text{(stump count)}$$

[3] Note that not all of them are *useful*, but we still have to count them.

Toward a learning algorithm for Decision Trees

Now let's count the DTs of size 2. Two possible DT structures can be obtained:



For each structure, there are $d(d-1)$ choices[4] for the attributes of the two non-leaf nodes and 2^3 ways to dispose the 2 class labels in the 3 leaf nodes.
This amounts to the following number of distinct DTs of size 2:

$$2 \times d(d-1) \times 2^3 = 2^4\, d(d-1) \qquad \text{(size-2 count)}$$

[4] By definition, each attribute can appear at most once in a path from root to leaf.

Toward a learning algorithm for Decision Trees

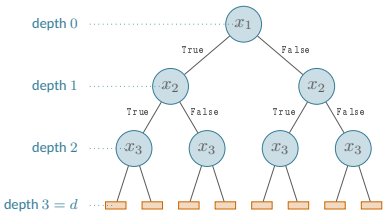
The largest DTs are **perfect binary trees**: every non-leaf node has 2 children and all the leaves are at depth d.

Let's count the non-leaf nodes:

- depth 0: $1 = 2^0$ node
- depth 1: $2 = 2^1$ nodes
- depth 2: $4 = 2^2$ nodes

$\vdots$

- depth $d-1$: 2^{d-1} nodes



$d = 3$: 7 non-leaf nodes, 8 leaves

Summing up:

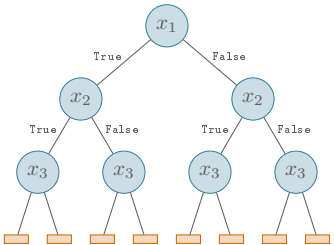
$1 + 2 + 4 + \cdots + 2^{d-1} = 2^d - 1$ non-leaf nodes (non-leaf count)

2^d leaves (leaf count)

Toward a learning algorithm for Decision Trees

How many distinct perfect DTs? Fill the nodes:

- non-leaf nodes: the 2^k nodes at depth k can each take any of the $d-k$ attributes not yet used in their path (*attribute placements*)
- leaves: each of the 2^d leaves takes one of the 2 class labels (*label assignments*)



$$\prod_{k=0}^{d-1}(d-k)^{2^k} = d \cdot (d-1)^2 \cdot (d-2)^4 \cdots 1^{2^{d-1}}$$ (attribute placements)

$$2^{2^d}$$ (label assignments)

$$2^{2^d} \cdot \prod_{k=0}^{d-1}(d-k)^{2^k}$$ (perfect DT count)

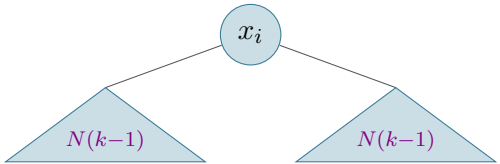
(e.g., $d=3$: $2^8 \cdot 3 \cdot 2^2 = 3072$).

Counting all the Decision Trees

Let $N(k)$ be the number of distinct DTs that can be built with k available attributes (each attribute at most once in a path, $m = 2$ classes). A tree is either:

`Spam` `Legitimate`

or



A single leaf:

2 choices of class label

A root with one of the k attributes and **two independent sub-trees** (True / False) built with the other $k-1$ attributes:

$k \cdot N(k-1)^2$ choices

Hence:

$$N(0) = 2 \quad N(k) = \underbrace{2}_{\text{leaf}} + \underbrace{k \cdot N(k-1)^2}_{\text{root + 2 sub-trees}}$$

(base case and recursion)

Counting all the Decision Trees

Check with $d = 2$ attributes:

With the recursion

$$N(2) = 2 + 2 \cdot N(1)^2 = 2 + 2 \cdot 6^2 = 74$$
$$N(1) = 2 + 1 \cdot N(0)^2 = 2 + 1 \cdot 2^2 = 6$$
$$N(0) = 2$$

(recursion)

By counting the trees

Trees with $d = 2$ attributes	**Number**
1 leaf (no non-leaf nodes)	2
size 1: decision stumps, $4d$	8
size 2: $2^4\, d(d-1)$	32
size 3: perfect trees	32
Total	**74**

The recursion gives the total for any d:

d	1	2	3	4	5
$N(d)$	6	74	16 430	$\approx 1.1 \cdot 10^9$	$\approx 5.8 \cdot 10^{18}$

Enumerating the DTs to find the smallest consistent one is not feasible. It's even worse, considering that these numbers are only for $m = 2$ classes.

Toward a learning algorithm for Decision Trees

The brute-force learning algorithm has a huge computational complexity.

It has been shown that finding the smallest consistent DT (e.g., by constructing all DTs of size 1, 2, etc., until a consistent one is found) is a **NP-hard problem**.

A naive learning algorithm for Decision Trees

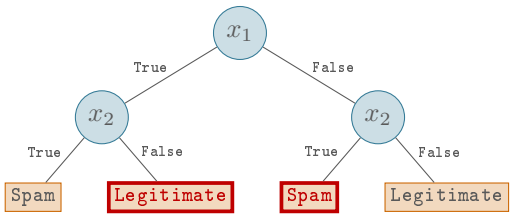
If we disregard the size, we have a very simple way to get a consistent DT.

1. for each training example $(\mathbf{x}, y)$:
   - build a distinct path from the root, testing all the d attributes (in any order), reusing the nodes of previous paths
   - end it with a leaf labeled y
2. complete each incomplete branch with a leaf with a random label

Note that this DT is **guaranteed** to correctly classify the samples.[5]

Example $(d = 2)$

	x_1	x_2	y
$\mathtt{m}_1$	True	True	Spam
$\mathtt{m}_2$	False	False	Legitimate



red: randomly labeled leaves

[5] Unless there are examples of different classes with identical attribute values.

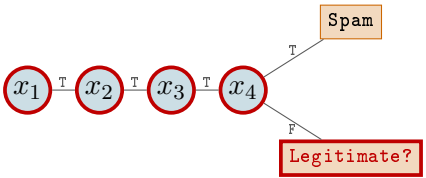
A naive learning algorithm for Decision Trees

The above learning algorithm builds a specific DT branch, i.e., a specific IF…THEN…classification rule, for each training example. In practice, the resulting DT just memorizes the training examples. The resulting DTs:

- are very likely over-fitting the training set
- tend to be much larger than necessary

A naive learning algorithm for Decision Trees

Counterexample. Consider a DT built with one single sample $\mathtt{m}_1 = (\mathtt{T}, \mathtt{T}, \mathtt{T}, \mathtt{T})$ labeled as `Spam`. Classify the unseen sample $\mathbf{x} = (\mathtt{T}, \mathtt{T}, \mathtt{T}, \mathtt{F})$, which is also spam:



(the other branches, e.g., $x_1 = \mathtt{False}$, also end in random leaves and are omitted)

Only the attribute combination of $\mathtt{m}_1$ reaches a leaf labeled by the training set. For all the other $2^4 - 1 = 15$ combinations, the naive DT outputs a **random** label. Here, $\mathbf{x}$ is misclassified with probability $1/2$.

Effective learning algorithms for Decision Trees: ID3

A trade-off is to find a “good enough” DT using some low-complexity heuristics that favor smaller DTs over larger ones.

This approach is obviously *suboptimal*. In the following, one of the first and simpler DT learning algorithms will be presented, named ID3 (Iterative Dichotomizer v. 3), by J.R. Quinlan (1986).[6]

[6] J.R. Quinlan, Induction of Decision Trees. *Machine Learning*, 1(1):81–106, 1986. https://doi.org/10.1023/A:1022643204877

The ID3 learning algorithm

ID3 is a **recursive, top-down** algorithm: it builds the DT from the root to the leaves.

For the training examples reaching an edge:

- all of the **same class** y: complete the edge with a leaf labeled y
- **different classes**: add a non-leaf node testing the most "discriminant" attribute, i.e., the one that best splits them into subsets of a same class

Sketch of the ID3 learning algorithm

Consider the training set T:

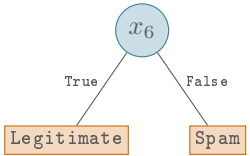
ID	y	x_1	x_2	x_3	x_4	x_5	x_6
m_1	L	T	T	T	T	F	T
m_2	L	T	F	T	F	T	T
m_3	L	T	T	F	T	F	T
m_4	L	F	F	F	T	T	T
m_5	L	F	F	T	F	F	T
m_6	L	T	T	T	F	T	T
m_7	S	F	T	T	T	F	F
m_8	S	F	T	F	T	F	F
m_9	S	T	F	F	F	T	F
m_{10}	S	F	T	T	T	T	F

Which attribute (*column*) is the most discriminant for T, to be used in the root node?

Sketch of the ID3 learning algorithm

- $x_6 = \texttt{True}$ (from m_1 to m_6): all **legitimate**
- $x_6 = \texttt{False}$ (from m_7 to m_{10}): all **spam**

x_6 yields the smallest consistent DT, a decision stump with two rules:



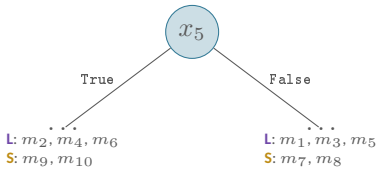
IF $x_6 = \texttt{True}$ THEN Legitimate IF $x_6 = \texttt{False}$ THEN Spam

Sketch of the ID3 learning algorithm

In real problems, no single attribute perfectly splits T.

Consider T **without** x_6. Starting from the root x_5:

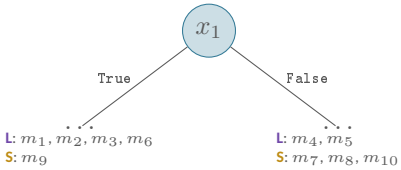
ID	y	x_1	x_2	x_3	x_4	x_5
m_1	L	T	T	T	T	F
m_2	L	T	F	T	F	T
m_3	L	T	T	F	T	F
m_4	L	F	F	F	T	T
m_5	L	F	F	T	F	F
m_6	L	T	T	T	F	T
m_7	S	F	T	T	T	F
m_8	S	F	T	F	T	F
m_9	S	T	F	F	F	T
m_{10}	S	F	T	T	T	T



- both subsets have no predominant class
- x_5 is a poor choice

Sketch of the ID3 learning algorithm

Root x_1:



- $x_1 =$ `True`: **L**:**S** = **4**:**1**, predominantly legitimate
- $x_1 =$ `False`: **L**:**S** = **2**:**3**, predominantly spam

(the evaluation of the other attributes is left as an exercise)

Both subsets still contain **different** classes: ID3 **recursively** builds a **sub-tree** for each edge, on the corresponding subset of T, using all the attributes **except** x_1.

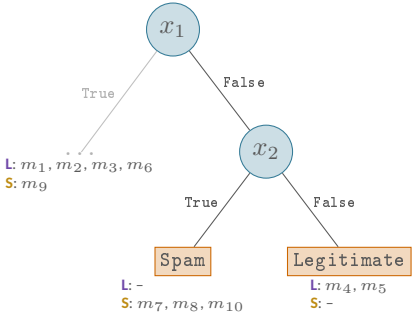
Sketch of the ID3 learning algorithm

Branch $x_1 = \texttt{F}$

Consider $m_4, m_5, m_7, m_8, m_{10}$ with all attributes except x_1.

ID	y	x_1	x_2	x_3	x_4	x_5
m_1	L	T	T	T	T	F
m_2	L	T	F	T	F	T
m_3	L	T	T	F	T	F
m_4	**L**	F	F	F	T	T
m_5	**L**	F	F	T	F	F
m_6	L	T	T	T	F	T
m_7	**S**	F	T	T	T	F
m_8	**S**	F	T	F	T	F
m_9	S	T	F	F	F	T
m_{10}	**S**	F	T	T	T	T

x_2 perfectly splits them, so both its branches are completed with a leaf:



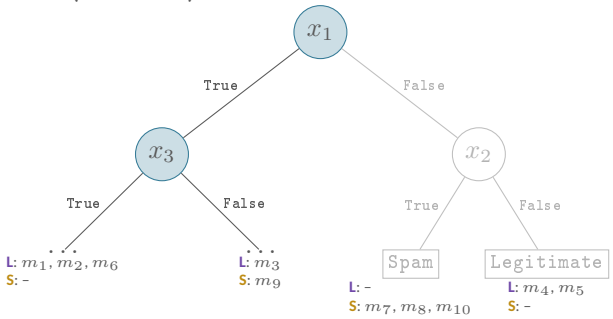
Sketch of the ID3 learning algorithm

Branch $x_1 = \texttt{T}$

Consider only m_1, m_2, m_3, m_6, m_9 , with all attributes except x_1.

ID	y	x_1	x_2	x_3	x_4	x_5
m_1	**L**	T	T	T	T	F
m_2	**L**	T	F	T	F	T
m_3	**L**	T	T	F	T	F
m_4	L	F	F	F	T	T
m_5	L	F	F	T	F	F
m_6	**L**	T	T	T	F	T
m_7	S	F	T	T	T	F
m_8	S	F	T	F	T	F
m_9	**S**	T	F	F	F	T
m_{10}	S	F	T	T	T	T

No attribute splits them perfectly, but x_3 is a good choice (isolates **L**).



Sketch of the ID3 learning algorithm

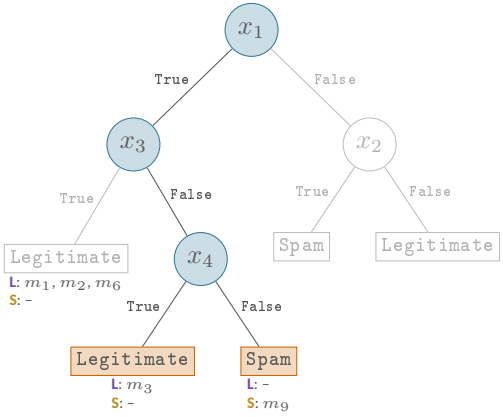
Branch $x_1 = \mathtt{T}$, $x_3 = \mathtt{F}$

Consider m_3 and m_9, with x_2, x_4, x_5 left.

ID	y	x_1	x_2	x_3	x_4	x_5
m_3	L	T	T	F	T	F
m_9	S	T	F	F	F	T

x_4 perfectly splits them, and the DT is complete (all samples are classified).

ID3 finds much smaller, consistent DTs than the naive algorithm, although it does not guarantee to find the smallest DT.



Sketch of the ID3 learning algorithm

Pseudo-code (the criterion for the most discriminant attribute is not specified yet)

T: training examples reaching the node;
A: attributes not yet used on the path from the root (initially all of $x_1, \ldots, x_d$).

```
1:  function DT_Learning(T, A)
2:      if T = ∅ or A = ∅ then                          ▷ no examples or no attributes left
3:          return leaf labeled majority_label(T)
4:      if T contains a single class y then             ▷ the subset is already pure
5:          return leaf labeled y
6:      x ← the most discriminant attribute in A        ▷ best split: criterion to be defined
7:      root ← new non-leaf node testing x              ▷ inner node
8:      for all value v of x do                         ▷ one branch per value
9:          T' ← {examples in T with x = v}             ▷ examples taking this branch
10:         subtree ← DT_Learning(T', A \ {x})          ▷ recurse, without x
11:         add to root an edge v leading to subtree
12:     return root
```

Evaluating attributes' discriminant capability

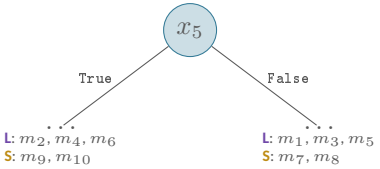
ID3's heuristic, for each non-leaf node n (root included):

- among the attributes not yet used in the branch, pick the one that splits the examples reaching n into subsets that are **as homogeneous as possible** (as close as possible to a single class)

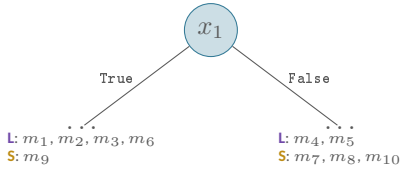
We need a **measure of homogeneity**: ID3 uses the **entropy** of a probability distribution.

Evaluating attributes' discriminant capability

Root node: the whole T (**6** : **4**) and all the attributes are considered.
Let's evaluate the following two possible choices:



$x_5 =$ T: **3** : **2**
$x_5 =$ F: **3** : **2**



$x_1 =$ T: **4** : **1**
$x_1 =$ F: **2** : **3**

Evaluating attributes' discriminant capability

For an attribute x, the **conditional probability** $P(Y \mid X = v)$ is the probability of class **L** (legitimate) or **S** (spam), given that $x = v$ (T: it contains the term, F: it does not).

It is estimated on each subset of the split as the fraction of its examples in each class, e.g.:

$$P(Y = \text{legitimate} \mid X_5 = \text{True}) = \frac{3}{5} = 0.6$$

For the three attributes considered so far:

	$P(\mathtt{L} \mid \mathtt{T})$	$P(\mathtt{S} \mid \mathtt{T})$	$P(\mathtt{L} \mid \mathtt{F})$	$P(\mathtt{S} \mid \mathtt{F})$
x_5	$3/5 = 0.6$	$2/5 = 0.4$	$3/5 = 0.6$	$2/5 = 0.4$
x_1	$4/5 = 0.8$	$1/5 = 0.2$	$2/5 = 0.4$	$3/5 = 0.6$
x_6	$6/6 = 1$	$0/6 = 0$	$0/4 = 0$	$4/4 = 1$

x_6 gives a **perfect** split: one class has probability 1 for each value.

Evaluating attributes' discriminant capability

For a given value v of x, with C classes:

- **best** discriminant capability (perfect split): $P(Y = y_v \mid X = v) = 1$ for one class y_v, and 0 for all the others
- **worst** discriminant capability (uniform split): $P(Y = y \mid X = v) = \frac{1}{C}$ for each class y

We need a **single value** that tells which attribute is the most discriminant.

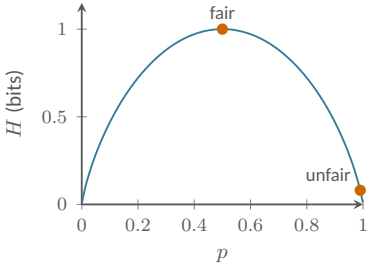
Evaluating attributes' discriminant capability

The **entropy** of a discrete random variable Y with domain $\mathcal{Y} = \{y_1, \ldots, y_C\}$ measures the uncertainty of its distribution (in bits):

$$H(Y) = -\sum_{i=1}^{C} P(Y = y_i) \log_2 P(Y = y_i) \text{ bits.}$$

Examples (coin with $P(\text{heads}) = p$):

- fair coin, $p = 0.5$ (maximum uncertainty):
  $H = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$ bit
- unfair coin, $p = 0.99$ (low uncertainty):
  $H = -(0.99 \log_2 0.99 + 0.01 \log_2 0.01) \approx 0.08$ bits



Evaluating attributes' discriminant capability

The entropy ranges in $H(Y) \in [0, \log_2 C]$:

- $H(Y) = \log_2 C$ if all values are equiprobable, $P(Y = y_i) = \frac{1}{C}$ (max. uncert.)
- $H(Y) = 0$ if one value is certain, $P(Y = y_j) = 1$ (no uncertainty)

For binary variables ($C = 2$):

$$H(Y) \in [0, \log_2 2] = [0, 1].$$

Evaluating attributes' discriminant capability

The **conditional entropy** of Y given X is the uncertainty about Y when the value of X is known. It is the average, over the values v of x, of the entropy of each subset:

$$H(Y \mid X) = \sum_{v} P(X = v)\, H(Y \mid X = v)\,,$$

where

$$H(Y \mid X = v) = -\sum_{i=1}^{C} P(Y = y_i \mid X = v) \log_2 P(Y = y_i \mid X = v)\,.$$

It is the ordinary entropy $H(Y)$ from the earlier slide, but using the class probabilities inside the subset where $X = v$.
Read it as: once you know $x = v$, you only look at the examples that reached that branch.

Evaluating attributes' discriminant capability

$H(Y \mid X)$ measures the discriminant capability of an attribute x:

- $H(Y \mid X) = 0$ if x splits the training set perfectly
- $H(Y \mid X) = \log_2 C$ if x splits it uniformly, for each of its values

ID3 chooses the attribute x with the **lowest** $H(Y \mid X)$ with respect to T.

Evaluating attributes' discriminant capability

In terms of Shannon's Information Theory, entropy is the expected amount of "missing" information (in bits):

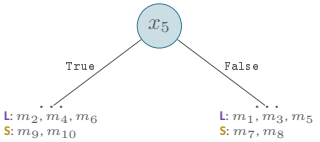
- $H(Y)$: the information missing to answer "what is the class label of a random instance?"
- $H(Y \mid X)$: the information still missing after observing the value of X

Choosing the attribute with the lowest $H(Y \mid X)$ therefore means maximizing the **information gain**, i.e., the information gained by the split:

$$\text{InformationGain} = H(Y) - H(Y \mid X)$$

Discriminant capability of attributes: examples

As an example, consider again the attribute x_5:



The conditional entropy of $P(Y \mid X_5)$ is:

$$H(Y \mid X_5) = \sum_{v \in \{\texttt{True},\texttt{False}\}} P(X_5 = v)\, H(Y \mid X_5 = v)\,.$$

First, since we have 5 training examples with $x_5 = \texttt{True}$ and 5 examples with $x_5 = \texttt{False}$, $P(X_5)$ can be estimated as:

$$P(X_5 = \texttt{True}) = P(X_5 = \texttt{False}) = \frac{5}{10} = 0.5\,.$$

Discriminant capability of attributes: examples

To compute $H(Y \mid X_5 = \texttt{True})$ and $H(Y \mid X_5 = \texttt{False})$, note first that there are 3 legitimate and 2 spam e-mails in T for both $x_5 = \texttt{True}$ and $x_5 = \texttt{False}$; therefore:

$$P(Y = \texttt{L} \mid X_5 = \texttt{True}) = 3/5 = 0.6\,,$$
$$P(Y = \texttt{S} \mid X_5 = \texttt{True}) = 2/5 = 0.4\,.$$

It follows that:

$$H(Y \mid X_5 = \texttt{True}) = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 \approx 0.97\,.$$

A similar computation leads to:

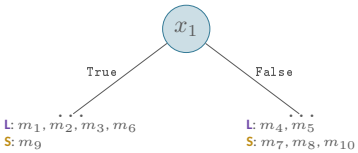
$$H(Y \mid X_5 = \texttt{False}) = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 \approx 0.97\,.$$

One finally obtains:

$$H(Y \mid X_5) \approx 0.5 * 0.97 + 0.5 * 0.97 = 0.97\,.$$

Discriminant capability of attributes: examples

Consider now the attribute x_1:



The conditional entropy of $P(Y \mid X_1)$ is:

$$H(Y \mid X_1) = \sum_{v \in \{\texttt{True},\texttt{False}\}} P(X_1 = v)\, H(Y \mid X_1 = v)$$

Also in this case there are 5 training examples with $x_1 = \texttt{True}$ and 5 with $x_1 = \texttt{False}$, therefore:

$$P(X_1 = \texttt{True}) = P(X_1 = \texttt{False}) = \frac{5}{10} = 0.5\,.$$

Discriminant capability of attributes: examples

To compute $H(Y \mid X_1 = \texttt{True})$, consider that there are 4 legitimate and 1 spam e-mail with $x_1 = \texttt{True}$, and therefore:

$$P(Y = \textsf{L} \mid X_1 = \texttt{True}) = 4/5 = 0.8\,,$$
$$P(Y = \textsf{S} \mid X_1 = \texttt{True}) = 1/5 = 0.2\,.$$

It follows that:

$$H(Y \mid X_1 = \texttt{True}) = -0.8 \log_2 0.8 - 0.2 \log_2 0.2 \approx 0.72\,.$$

To compute $H(Y \mid X_1 = \texttt{False})$, noting that there are 2 legitimate and 3 spam e-mail with $x_1 = \texttt{False}$, one obtains:

$$P(Y = \textsf{L} \mid X_1 = \texttt{False}) = 2/5 = 0.4\,,$$
$$P(Y = \textsf{S} \mid X_1 = \texttt{False}) = 3/5 = 0.6\,.$$

Discriminant capability of attributes: examples

One then gets:

$$H(Y \mid X_1 = \texttt{False}) = -0.4 \log_2 0.4 - 0.6 \log_2 0.6 \approx 0.97,$$

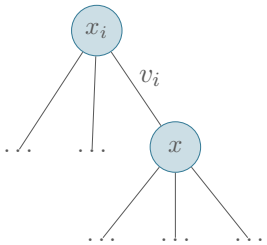
which finally leads to:

$$H(Y \mid X_1) \approx 0.5 \times 0.72 + 0.5 \times 0.97 = 0.845.$$

Note that $H(Y \mid X_1) < H(Y \mid X_5)$; therefore, x_1 is more discriminant than x_5 for the root node of a DT.

Evaluating attributes' discriminant capability

The discriminant capability of attributes at inner nodes can be computed similarly.
If x_i has been chosen, and a different attribute $x \neq x_i$ is being evaluated:

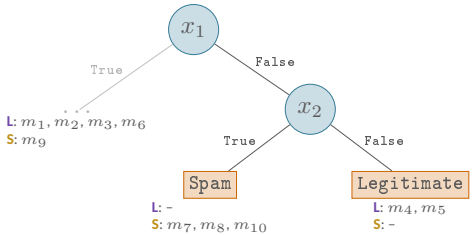


In this case only the training examples for which $x_i = v_i$ have to be considered.

$$H(Y \mid X, X_i = v_i) = \sum_v P(X = v \mid X_i = v_i)\, H(Y \mid X = v, X_i = v_i)\ .$$

Discriminant capability of attributes: examples

Assume that x_1 has been chosen for the root, and consider x_2 on the edge $x_1 = \texttt{False}$:



In this case, only $\mathtt{m}_4$, $\mathtt{m}_5$, $\mathtt{m}_7$, $\mathtt{m}_8$ and $\mathtt{m}_{10}$ have to be considered, i.e., 2 **L** and 3 **S**.
The corresponding conditional entropy is:

$$H(Y \mid X_2, X_1 = \mathtt{F}) = \sum_{v \in \{\mathtt{T},\mathtt{F}\}} P(X_2 = v \mid X_1 = \mathtt{F})\, H(Y \mid X_2 = v, X_1 = \mathtt{F})$$

Discriminant capability of attributes: examples

$$P(X_2 = \texttt{True} \mid X_1 = \texttt{False}) = 3/5 = 0.6\,,$$
$$P(X_2 = \texttt{False} \mid X_1 = \texttt{False}) = 2/5 = 0.4\,.$$

Since x_2 perfectly splits the considered training examples, one gets:

$$P(Y = \texttt{L} \mid X_2 = \texttt{True}, X_1 = \texttt{False}) = 0\,,$$
$$P(Y = \texttt{S} \mid X_2 = \texttt{True}, X_1 = \texttt{False}) = 1\,,$$

and therefore $H(Y \mid X_2 = \texttt{True}, X_1 = \texttt{False}) = 0$.
Similarly, $H(Y \mid X_2 = \texttt{False}, X_1 = \texttt{False}) = 0$.
It easily follows that the conditional entropy of x_2, **in this node**, equals 0:

$$H(Y \mid X_2, X_1 = \texttt{False}) = 0.6 \times 0 + 0.4 \times 0 = 0\,.$$

Evaluating attributes' discriminant capability

To sum up, for each non-leaf node n, ID3 evaluates every candidate attribute x by its conditional entropy, computed:

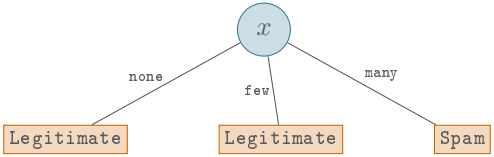
- only on the training examples that reach n
- given the values of the attributes already tested in the same branch

ID3 chooses the attribute with the lowest conditional entropy.

Other algorithms use other measures, e.g., the **Gini index**, by the statistician Corrado Gini (1884–1965), originally a measure of income inequality.

Evaluating attributes' discriminant capability

So far all the attributes were Boolean, hence two branches. In general, a non-leaf node has **one branch per value** of its attribute: an attribute with k values gives k children.

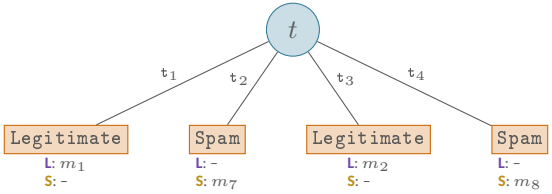


E.g., x = how often the term `cheap` appears in an e-mail: none, few, many ($k = 3$).
The entropy formulas do not change.[7]

[7] The sums already run over **all** the values v of x.

Evaluating attributes' discriminant capability

The conditional entropy fails for attributes that are **irrelevant** or have **many values**.
Example: The time t at which an e-mail is received is irrelevant, but in the training set it has a different value for each e-mail.



Each branch holds a single example, so: $H(Y \mid T) = \sum_{i=1}^{4} \frac{1}{4} 0 = 0$
The time seems the best attribute for the root, but it **does not generalize**.

Evaluating attributes' discriminant capability

To avoid this, ID3 actually uses the **gain ratio**: the information gain is **divided by a penalty** for splitting into many branches.

$$\text{GainRatio} = \frac{\text{InformationGain}}{\text{SplitInfo}}$$

SplitInfo is the entropy of the split itself: the bits needed to tell **which branch** an example goes to (p_i: fraction of the N examples going to the i-th of the v branches).

$$\text{SplitInfo} = -\sum_{i=1}^{v} p_i \log_2 p_i \qquad p_i = \frac{N_i}{N}$$

- 2 equal branches: one yes/no question ("left or right?") $\Rightarrow$ 1 bit;
- 4 equal branches: 2 questions $\Rightarrow$ 2 bits;

Examples spread evenly over **many** values $\Rightarrow$ **large** SplitInfo $\Rightarrow$ **low** gain ratio.

Evaluating attributes' discriminant capability

For v equal branches, each branch receives a fraction $p_i = \frac{1}{v}$ of the examples:

$$
\begin{aligned}
\text{SplitInfo} &= -\sum_{i=1}^{v} p_i \log_2 p_i \\
&= -\sum_{i=1}^{v} \frac{1}{v} \log_2 \frac{1}{v} \\
&= -v \cdot \frac{1}{v} \log_2 \frac{1}{v} \qquad \text{(the } v \text{ terms are all equal)} \\
&= -\log_2 \frac{1}{v} \\
&= \log_2 v
\end{aligned}
$$

Evaluating attributes' discriminant capability

SplitInfo for 8 samples spread over the values of an attribute (samples per value in parentheses):

Evenly spread

$$(4,4):\ -2\,\tfrac{4}{8}\log_2\tfrac{4}{8} = \mathbf{1}$$
$$(2,2,2,2):\ -4\,\tfrac{2}{8}\log_2\tfrac{2}{8} = \mathbf{2}$$
$$(1,\ldots,1):\ -8\,\tfrac{1}{8}\log_2\tfrac{1}{8} = \mathbf{3}$$

Unevenly spread

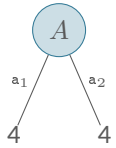
$$(7,1):\ -\tfrac{7}{8}\log_2\tfrac{7}{8} - \tfrac{1}{8}\log_2\tfrac{1}{8} \approx \mathbf{0.54}$$
$$(5,1,1,1):\ -\tfrac{5}{8}\log_2\tfrac{5}{8} - 3\,\tfrac{1}{8}\log_2\tfrac{1}{8} \approx \mathbf{1.55}$$

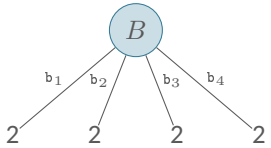
v equal branches: $\log_2 v$ bits

The more branches, and the more evenly the examples are spread over them, the larger the SplitInfo.

Evaluating attributes' discriminant capability

Two attributes, with the 8 samples equally distributed among their values (the numbers are the samples per value). Suppose they have the same information gain, e.g., 0.5 bits:





$$\text{SplitInfo}(A) = -2\,\tfrac{4}{8}\log_2\tfrac{4}{8} = 1 \qquad \text{SplitInfo}(B) = -4\,\tfrac{2}{8}\log_2\tfrac{2}{8} = 2$$

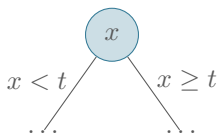
$$\text{GainRatio}(A) = \frac{0.5}{1} = \mathbf{0.5} \qquad \text{GainRatio}(B) = \frac{0.5}{2} = \mathbf{0.25}$$

To reduce the number of splits for numerical attributes, ID3 uses **binary splits** (next).

Extensions of ID3 to numerical attributes

A **numerical** attribute x may have many values (e.g., the number of edge pixels in an image), or infinitely many (e.g., continuous).
One edge per value would be impractical, may cause over-fitting, or is not even possible.

ID3 always uses a **binary split** with a **threshold** t:



Extensions of ID3 to numerical attributes

To choose t, ID3 considers only the q different values $x_{(1)} < x_{(2)} < \cdots < x_{(q)}$ that x takes among the training examples reaching the node, and tests the $q-1$ **midpoints**:

$$t_k = \frac{x_{(k+1)} + x_{(k)}}{2}\,, \quad k = 1, \ldots, q-1\,.$$

Example: values $\{1, 3, 4, 8\} \Rightarrow$ thresholds $t \in \{2,\ 3.5,\ 6\}$.

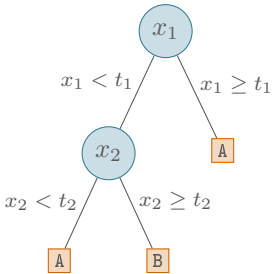
For each t_k, the tests $x < t_k$ and $x \geq t_k$ split the examples: ID3 chooses the t_k with the **minimum conditional entropy**.

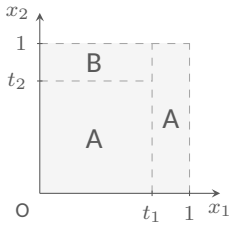
Decision regions for numerical attributes

A classifier $h : X \to Y$ maps the attribute space to the labels.

The parts of the space with the same label are its **decision regions**, which can be plotted if $d \leq 3$ (each one is an axis).

Example: labels A and B, $d = 2$, and $x_1, x_2 \in [0, 1]$.





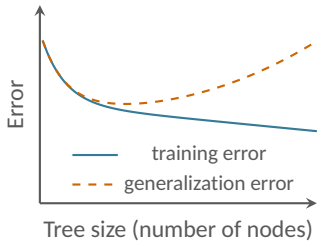
The boundaries are **lines parallel to the axes** of the attribute space.

Over-fitting in Decision Trees

Consistency can force ID3 to build overly-specialized DTs that don't generalize well. This condition is known as **over-fitting**.

Over-fitting can be spotted by plotting the error as a function of the DT size, for a fixed training set:

- **training** error keeps decreasing
- **generalization** error (on a different set of labeled examples) decreases, then **increases**



Mitigating over-fitting: Decision Tree pruning

Pruning removes deep branches devoted to a few examples (at the cost of consistency).

- **offline**: build a consistent DT, then replace some sub-trees with a leaf
- **online**: stop growing a branch when a criterion is met, and add a leaf

The leaf takes the label of the **majority class** among the training examples reaching it, to minimize training errors.

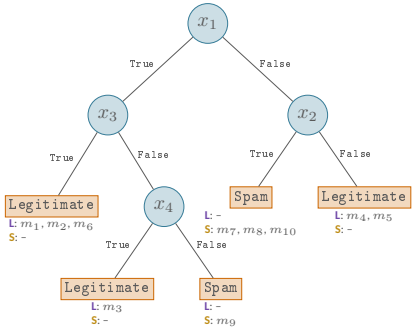
Some pruning criteria (offline or online):

- a maximum tree depth
- a minimum number of training examples reaching a node
- the splitting criterion of the best attribute (e.g., gain ratio, Gini index) is too low

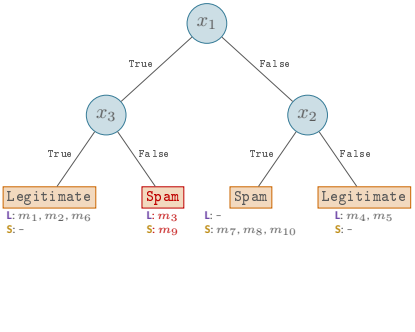
Mitigating over-fitting: Decision Tree pruning

Prune the DT by removing the **sub-trees reached by less than 3** training examples.

Full DT



Pruned DT



The sub-tree of x_4 is reached by one **L** and one **S** e-mail only, so it is replaced by a leaf. With a tie, the label is chosen randomly, e.g., Spam.